AI Bootcamp — Week 1: Foundations

Theme: Social Media Content Creation

© 2026 Improving

TIMING

- **Monday:** 1:00 PM – 5:00 PM (travel morning, 4 hours)
- **Tuesday–Thursday:** 9:00 AM – 5:00 PM with 1-hour lunch (7 hours each)
- **Friday:** 9:00 AM – 12:00 PM (travel afternoon, 3 hours)

© 2026 Improving

OBJECTIVES

By end of Week 1, the candidate can:

- Explain the Prompt/Harness/Model framework and why the harness, not the model, controls what a skill can do
- Apply the EDD (Evaluation-Driven Development) discipline: write Evaluation Criteria (EC) and manual test cases before building any skill
- Explain and apply the RTCC prompt structure (Role = WHO, Task = WHAT, Context = WHY, Constraints = HOW) to a real skill
- Build four sequential content creation skills as durable, committed `.skill.md` files
- Direct an AI to write a Python evaluation script (deterministic checks plus an LLM judge) and interpret its results
- Verify that every instruction in a skill is load-bearing using manual tests and evaluation runs
- Produce a working end-to-end content pipeline: Trend Identification → Ideation → Development → Graphics

© 2026 Improving

THE WEEK 1 PIPELINE

All four skills in this week feed into each other. Each produces a durable Markdown file that becomes the input to the next step.

```
Trend Identification  →  Ideation  →  Development  →  Graphics
(trends.md)              (ideas.md)   (post.md)        (graphics.md)
```

Each pipeline stage is built as a **skill file** (`.skill.md`) with YAML frontmatter and `disable-model-invocation: true` , invoked explicitly via slash command. You choose when it runs; nothing triggers it for you. (Week 3 covers when and why that flag ever changes.)

The theme, Social Media Content Creation, is chosen deliberately: it has no right answer, the quality criteria are clear enough to test, and it is entirely disconnected from any client codebase. This lets candidates focus on the discipline of EDD and RTCC without the distraction of domain-specific technical work.

Stretch practice: if you finish an exercise early, pick a kata from `Resources/Katas/` (Requirements + Expected Outcomes pairs) and run the same EDD discipline against it. Katas are optional reps, not deliverables.

© 2026 Improving

MONDAY (1:00 PM – 5:00 PM): EDD, PROMPT/HARNESS/MODEL, AND RTCC

1:00 – 2:15 PM — EDD: DEFINE SUCCESS BEFORE YOU BUILD

Core concept: The single most important discipline in this program is writing your tests before writing your prompt. Every skill you build this week follows this order:

1. Write the manual test file (Input Prompts + Expected Outputs) by hand. The Expected Outputs are your **Evaluation Criteria (EC)**: the quality bar for the skill's output.
2. Run the test against a baseline prompt. Some tests should fail.
3. Write the skill to make failing tests pass.
4. Verify every instruction is load-bearing; remove any that are not.

The Load-Bearing Rule: If a test passes without an instruction you're about to add, do not add it. Prompts are not documentation. Every word should be doing work that can be proven.

Why manual, not AI-generated test files: When you manually define Expected Outputs, you commit to what success looks like before the model has shaped your expectations. If you let the model generate the test cases, you may end up testing that the model produces what the model would produce anyway, which tells you nothing about whether your instructions are actually altering behavior.

Exercise: Given this bare prompt, `"Identify trending topics in AI on social media"`, write a manual test file with 5 entries. For each entry, specify:

- **Input:** a specific focus/realm the user would provide
- **Expected Output (EC):** describe the *structure and content criteria* the response must meet to pass (not the exact text, the criteria)

Examples of bad EC: *"A list of good topics."* Examples of good EC: *"A numbered list of 5–8 topics. Each entry includes: the topic name, one sentence explaining why it is currently trending, and 3–5 associated hashtags in #format. No topic should be more than 3 months old based on training context. No entry should repeat another."*

Spend 30 minutes writing these 5 test cases. Then run the bare prompt against each. Record which pass and which fail. You should see failures. That is the point.

2:15 – 3:00 PM — PROMPT/HARNESS/MODEL

Before going further, get the conceptual frame that the entire DLP is built on:

- **Prompt:** the instructions you give to a model. What you want it to do, how you want it done, what constraints it must follow.

© 2026 Improving

- **Harness:** the software environment that mediates between you and the model. The harness determines what the model can actually do: read files, run commands, access tools. Claude Code, Devin, GitHub Copilot, and ChatGPT web are all different harnesses with different boundaries.
- **Model:** the AI itself. It generates text based on its training and the prompt, but it cannot do anything on your machine that the harness does not allow.

A model without a harness is a chat interface. The harness is the capability boundary; the model is the intelligence; the prompt is the instruction.

Why this matters this week: the skills you are building carry `disable-model-invocation: true` in their frontmatter. That flag is a harness-level decision. It means the harness will never let the model decide to run your skill on its own; it only runs when you invoke it. You keep invocation control while you build trust in the skill through evaluation evidence. (Week 3 covers the graduation criteria for flipping that flag.)

Discussion: which harness are you running? What can it do that a chat window cannot? What can it not do?

3:00 – 4:15 PM — RTCC: THE PROMPT STRUCTURE

RTCC is the four-part prompt structure used throughout this program:

- **R — Role (WHO):** who the model should act as. A specific, credentialed persona changes tone, vocabulary, and default assumptions.
- **T — Task (WHAT):** the specific action it should perform. Verb-first and unambiguous.
- **C — Context (WHY):** the intent behind the task. Why is this being done, what is it for, what does a good outcome make possible?
- **C — Constraints (HOW):** the data, rules, and specifications the model needs. What the output must contain, what it must never do, what format is required.

Why this split matters: Context is not "background information." It is the *why*. Constraints are not "extra rules." They are the *how*. When an instruction feels out of place during a load-bearing audit, the first diagnostic is structural: is this instruction in the right RTCC section? A rule sitting in Context will be treated as rationale, not enforced as a requirement.

Why this order matters: Role and Task together define the basic contract. Context shapes how the contract is fulfilled. Constraints are guardrails. Adding Constraints before establishing Role and Task often produces over-restricted outputs that satisfy the guardrails but miss the intent.

Exercise: Rewrite the bare trend-identification prompt from earlier as a full RTCC prompt. Draft each element separately before combining:

- R: Who should be doing this identification?
- T: What exactly are they doing?
- C (WHY): Why does this identification matter? What will the output be used for, and by whom?
- C (HOW): What must be true of the output, and what should it never include?

© 2026 Improving

After combining: re-run your 5 manual test cases. How many now pass? For any that still fail, identify which RTCC element is missing or weak.

4:15 – 5:00 PM — REFLECTION

- Review your test file and your RTCC prompt side by side.
- For each instruction in the prompt, remove it mentally: would any test fail? If no test would fail, flag the instruction for removal or replacement.
- For each flagged instruction: is it in the right RTCC section, or is it a Constraint masquerading as Context?
- Discuss: What was hard to define in Expected Output? What made an EC too vague to test against?

© 2026 Improving

TUESDAY (9:00 AM – 5:00 PM): TREND IDENTIFICATION SKILL

9:00 – 10:00 AM — THE PIPELINE AND SKILL 1 BRIEF

Overview of the full week pipeline. Each skill produces a durable Markdown file with a defined schema. The instructor demonstrates a complete pipeline run so candidates understand what they are building toward.

Skill 1: Trend Identification:

- **Input:** A topic area or realm (e.g., "AI in software development", "sustainable fashion", "Formula 1")
- **Output file:** `trends.md` : a structured list of current trending topics and hashtags within that realm
- **Required output elements:** Topic name, why it's trending (one sentence, evidence-based or model-knowledge-based), 3–5 hashtags per topic in `#format` , platform relevance notes (LinkedIn vs. X/Twitter vs. Instagram differ)

Skill file anatomy. A skill file is a Markdown file with YAML frontmatter:

```
---
name: trend-identification
description: Identify current trending topics and hashtags within a given realm
disable-model-invocation: true
---
```

The body below the frontmatter is your RTCC prompt. `disable-model-invocation: true` means the harness runs this skill only when you invoke it explicitly (via slash command). The `description` tells the harness, and your future self, what the skill is for.

10:00 – 11:30 AM — EDD FIRST: WRITE THE TEST FILE

Before opening your IDE, write the manual test file for the Trend Identification skill. Do this on paper or in a plain text file, not in the IDE, not with AI assistance.

For at least 3 different topic areas, define:

- The exact input you will provide
- The EC the `trends.md` output must meet to pass (structure, content, length, format, freshness signals)
- At least one explicit failure criterion: something the output must NOT contain or do

Commit this test file before writing a single line of the skill. The test file is your Red state.

11:30 AM – 1:00 PM — BUILD THE TREND IDENTIFICATION SKILL

Write the Trend Identification skill as an RTCC-structured `.skill.md` file with the frontmatter above. Apply the load-bearing rule: every sentence you add should correspond to at least one EC. If you're adding language that no EC would catch, either add the EC or don't add the language.

Commit the skill file.

© 2026 Improving

1:00 – 2:00 PM — LUNCH

2:00 – 3:30 PM — RUN AND ITERATE

Run the skill against your test inputs. Grade each output against your EC. Record results in the test file (pass/fail per criterion per input). Identify the weakest criterion. Form a specific hypothesis: *"The output is failing criterion X because the prompt does not specify Y."* Make one targeted change. Re-run. Repeat at least twice.

Rule: One change per iteration. Document the hypothesis before making the change.

3:30 – 5:00 PM — AI-WRITTEN PYTHON EVALUATION SCRIPTS

Manual grading taught you what your EC actually mean. Now you need a measurement tool you can run on every iteration for the rest of this program: a **Python evaluation script**.

The key discipline: **the AI writes the code; you direct and review.** There is no Python prerequisite. Your job is to bring the EC (which you wrote by hand, before the skill existed) and to read what the AI produces closely enough to explain every check it contains. The models carry the coding load.

Two kinds of checks, in this order:

1. **Deterministic checks first.** Schema, counts, format, structure. These run without any model call: fast, free, definitive. Examples for this skill: does `trends.md` contain 5–8 numbered topics? Does each topic have 3–5 hashtags in `#format`?
2. **LLM judge second.** For EC that genuinely require qualitative judgment, the script sends the skill output plus the EC text to a model and gets back a structured JSON judgment: pass/fail, score, and reasoning per criterion. The rubric text should be the same EC you wrote in your manual test file. If your manual EC were vague, the judge's verdicts will be vague too. This is the feedback loop.

> *If you find yourself using an LLM judge for something that could be a regex, your EC needs refinement.*

Exercise:

- Direct the AI to write an evaluation script for your Trend Identification skill: at least 2 deterministic checks and 1 LLM judge check, driven by your manual test file.
- Read the script. Be able to explain what each check does and why it is deterministic or judged. (The certification gate requires you to *demonstrate* this, not describe it.)
- Run the script against your current skill's output. Record the baseline result. This is your Red state for tomorrow.

Commit the evaluation script alongside the skill it tests.

© 2026 Improving

WEDNESDAY (9:00 AM – 5:00 PM): IDEATION SKILL

9:00 – 10:00 AM — EDD FIRST: WRITE THE IDEATION TEST FILE

Skill 2: Ideation:

- **Input:** The `trends.md` file from Skill 1 (or a specified trend from it)
- **Output file:** `ideas.md`: a structured list of post ideas for social media, each grounded in one of the identified trends
- **Required output elements:** Post concept (one-line summary), target platform, estimated audience segment, angle or hook, why this trend makes this idea timely now

Write your manual test file before touching the skill. For at least 3 inputs (different trends or trend files), define the EC and at least one explicit failure mode.

10:00 – 11:30 AM — BUILD AND EVALUATE FROM THE START

For the Ideation skill, skip the manual-only phase and bring evaluation in from the first build. Write the RTCC skill, then immediately direct the AI to generate the evaluation script from your EC (deterministic checks first, LLM judge where judgment is required). Run the baseline (Red state), then iterate.

Every change: one hypothesis, one instruction change, one evaluation run.

11:30 AM – 1:00 PM — LUNCH

1:00 – 3:00 PM — REFINEMENT AND THE LOAD-BEARING AUDIT

After at least 3 evaluation iterations on the Ideation skill, perform the load-bearing audit:

- Read each instruction in the skill
- For each: predict which check would fail if you removed it
- If you cannot name a check that would fail, flag the instruction
- Remove flagged instructions and run the evaluation again: confirm the results do not drop

Candidates consistently find instructions that "feel important" but are load-bearing on nothing. This is the refactor step, and the evaluation script makes it mechanical.

3:00 – 5:00 PM — PIPELINE CONNECTION: TREND ID → IDEATION

Run the two-skill pipeline end to end:

1. Run Trend Identification on a topic area → produce `trends.md`
2. Feed `trends.md` as input to Ideation → produce `ideas.md`

© 2026 Improving

Evaluate the `ideas.md` output against your Ideation EC. Note: some failures may be caused by weak `trends.md` output, not by the Ideation skill itself. Trace failures to their source.

This is the first time candidates encounter **upstream quality dependency**, a concept that recurs throughout the program.

© 2026 Improving

THURSDAY (9:00 AM – 5:00 PM): DEVELOPMENT SKILL

9:00 – 10:00 AM — EDD FIRST: WRITE THE DEVELOPMENT TEST FILE

Skill 3: Development:

- **Input:** A selected post idea from `ideas.md`
- **Output file:** `post.md`: a fully developed social media post ready for review and scheduling
- **Required output elements:** Post body (platform-appropriate length), hashtags (platform-appropriate count and style), call to action, target platform(s), audience segment, content warnings or tone notes if applicable

Define your EC before building. Be specific about what "platform-appropriate" means for each platform you test against: LinkedIn is not X/Twitter is not Instagram. These are separate, testable criteria.

10:00 AM – 1:00 PM — BUILD, EVALUATE, FULL EDD CYCLE

Build the Development skill. Have the AI generate the evaluation script from your EC before the first run. Full Red/Green/Refactor cycle. Minimum: 4 evaluation runs with documented hypotheses. After achieving Green, perform the load-bearing audit.

1:00 – 2:00 PM — LUNCH

2:00 – 4:00 PM — THREE-SKILL PIPELINE RUN

Run the full pipeline: Trend Identification → Ideation → Development. Produce a complete `post.md` from a raw topic area. Evaluate the end-to-end quality.

Discussion: Where in the pipeline did quality degrade most? What would you fix first and why?

4:00 – 5:00 PM — REFLECTION: WHAT MAKES AN INSTRUCTION EARN ITS PLACE

Group discussion. Each candidate picks the instruction they are most surprised turned out to be load-bearing, and the one they were most surprised turned out to be dead weight. What does this tell you about how prompts work?

© 2026 Improving

FRIDAY (9:00 AM – 12:00 PM): GRAPHICS SKILL AND PIPELINE COMPLETION

9:00 – 10:00 AM — EDD FIRST: WRITE THE GRAPHICS TEST FILE

Skill 4: Graphics:

- **Input:** A completed `post.md`
- **Output file:** `graphics.md` : 3 to 5 fully formed image generation prompts suitable for Midjourney, DALL-E, or equivalent
- **Required output elements per prompt:** Subject description, visual style/aesthetic, color palette notes, mood/tone, composition guidance, aspect ratio recommendation for the target platform, negative prompt suggestions (what to exclude)

Write your test file. For "good" image generation prompts, the EC should be objective enough to evaluate without running the prompt through an image generator. Think: would a designer reading this prompt know exactly what to produce?

10:00 – 11:00 AM — BUILD, EVALUATE, AND LOAD-BEARING AUDIT

Build the Graphics skill. Have the AI generate the evaluation script. Run the full EDD cycle. Perform the load-bearing audit before declaring done.

11:00 AM – 12:00 PM — FULL PIPELINE RUN AND WEEK 1 WRAP

Run the complete four-skill pipeline from a single topic input to a complete `graphics.md`. Review all four output files for quality and coherence.

Week 1 Artifacts Review:

- [ ] `trend-identification` skill (RTCC, committed `.skill.md` with `disable-model-invocation: true`, all instructions load-bearing)
- [ ] `ideation` skill (same standard)
- [ ] `development` skill (same standard)
- [ ] `graphics` skill (same standard)
- [ ] Manual test files (EC) for at least Trend Identification (hand-written, pre-build)
- [ ] Python evaluation scripts and run results for all four skills (deterministic checks plus LLM judge)
- [ ] Load-bearing audit documentation for at least two skills

© 2026 Improving

WEEK 1 DELIVERABLES (COMMITTED TO REPO)

- [ ] Four RTCC skill files (one per pipeline stage), all durable `.skill.md` files with `disable-model-invocation: true`
- [ ] Manual test file for Trend Identification, written before the skill was built
- [ ] Python evaluation scripts and baseline + iteration results for all four skills
- [ ] Load-bearing audit notes: flagged instructions, removed instructions, confirmed failures
- [ ] Sample pipeline run: `trends.md` → `ideas.md` → `post.md` → `graphics.md` (one complete run, committed)

© 2026 Improving

COACHING NOTES FOR INSTRUCTORS

Common failure modes:

- **Candidates write EC that are too vague to test against.** Push them to get specific before they start building. "Contains a list of topics" is not a criterion. "Contains 5–8 topics, each with a trend rationale and 3–5 hashtags" is a criterion.
- **Candidates write test cases after building the skill.** This inverts EDD and produces tests that confirm what the model already does rather than defining what it should do. Enforce the commit order: test file committed before skill file.
- **Candidates let the AI write the EC along with the script.** The AI writes the evaluation *code*, never the criteria. The EC come from the manual test file, written before the skill existed. If the AI is defining what "good" means, the feedback loop is broken.
- **Candidates add instructions without a failing test to justify them.** The load-bearing audit at the end of each skill is not optional. Dead instructions are the enemy of reliable prompts.
- **Candidates conflate "it sounds like good output" with "it meets the EC."** Scoring against the EC must be structured. If a criterion is ambiguous enough that two people would grade it differently, the criterion needs to be rewritten.
- **Everything becomes an LLM judge.** If the evaluation script has six judge calls and no deterministic checks, the EC are not sharpened yet. Counts, formats, and required sections are regexes and schema checks: fast, free, and consistent. Push candidates to convert judge checks into deterministic ones wherever possible.
- **LLM judge rubrics that are too permissive.** If a judge check passes trivially, the rubric text is not specific enough. Require candidates to show at least one judge check that fails before they consider a skill done.
- **Candidates never read the generated script.** The whole point of AI-written evaluation code is that a non-Python user can own it. If a candidate cannot explain what each check does, they are trusting output they have not verified, which is the over-truster failure mode this program exists to eliminate.

© 2026 Improving